System Verification and Validation Plan for FPVR

Team 9, Software Mandems
John Cho
Ibrahim Tariq
Vikram Chandar
Sankalpa Chhetri Dhakal
Arjun Sasidaran

November 3, 2026

Revision History

Date	Version	Notes
November 2, 2026	1.0	Initial version of the VnV Plan

Contents

1 Symbols, Abbreviations, and Acronyms **vi**

2 General Information **1**

2.1 Summary 1

2.2 Objectives 1

2.3 Custom Documents 2

2.4 Relevant Documentation 2

3 Plan **3**

3.1 Verification and Validation Team 5

3.2 SRS Verification 6

3.2.1 Checklist Inspection 6

3.2.2 Supervisor Review 7

3.2.3 Peer and TA Review 7

3.2.4 Traceability Review 7

3.2.5 Continuous Verification 7

3.3 Design Verification 8

3.4 Verification and Validation Plan Verification 9

3.5 Implementation Verification 9

3.6 Automated Testing and Verification Tools 11

3.7 Software Validation 12

4 System Tests **13**

4.1 Tests for Functional Requirements 13
4.1.1 Realistic Flight Simulation (S.2.1) 13
4.1.2 Menu Interface (S.2.2, S.2.5) 16
4.1.3 Checkpoint and Crash Recovery (S.2.3) 22
4.1.4 Timing and Performance Evaluation (S.2.4) 25
4.1.5 Player Accounts and Authentication (S.2.6) 28
4.1.6 Result Storage and Leaderboards (S.2.7) 30
4.1.7 Post-Flight Review and Intelligent Feedback (S.2.8, S.2.9) 34
4.2 Tests for Nonfunctional Requirements 36
4.2.1 Installability and Compatibility 37
4.2.2 Performance 38
4.2.3 Reliability 39
4.2.4 Usability 40
4.2.5 Safety 41
4.2.6 Security and Privacy 43
4.2.7 Data Persistence and Offline Availability 45
4.3 Traceability Between Test Cases and Requirements 46

5 Unit Test Description **48**

5.1 Unit Testing Scope 49
5.2 Tests for Functional Requirements 49
5.2.1 Module 1 49
5.2.2 Module 2 50
5.3 Tests for Nonfunctional Requirements 50

5.3.1 VR Interface and Input 50
5.3.2 Checkpoint Progress and Recovery 51
5.3.3 Accounts and Online Service 51
5.4 Traceability Between Test Cases and Modules 52

6 Appendix **53**
6.1 Symbolic Parameters 53
6.2 Usability Survey Questions 55
6.2.1 Participants and Evaluation Procedure 56
6.2.2 Participant Background 57
6.2.3 Usability Evaluation Questions 57
6.2.4 Additional Feedback Questions 59
6.2.5 Evaluation and Analysis 59

7 Appendix — Generative AI Use Disclosure **61**
7.1 AI Tools Used 61
7.2 How and Where Used 61
7.3 Verification of AI-Generated Content 63

8 Appendix — Reflection **66**

List of Tables

1 Symbols, abbreviations, and acronyms vii
2 V&V activities by milestone 4

3 Area reviewer rotation 6
4 SRS verification checklist 8
5 Design verification checklist 9
6 V&V plan verification checklist 10
7 Implementation verification checklist 11
8 Required FPVR test configurations 12
9 Traceability between functional requirements and system tests 47
10 Traceability between nonfunctional requirements and system tests 48
11 Symbolic Constants for System Tests 54
12 Additional Safety and Validation Parameters 55

List of Figures

1 Symbols, Abbreviations, and Acronyms

The symbols, abbreviations, and acronyms used in this document are listed below. Each abbreviation in the text links back to its entry in this table.

Table 1: Symbols, abbreviations, and acronyms

Term	Description
C	Number of registered crashes in an attempt, used in the score
T	Total active flight time of an attempt in seconds, used in the score
AI	Artificial intelligence, used for the optional intelligent feedback
CI	Continuous integration
FPS	Frames per second
FPV	First-person view: presentation from the simulated drone's viewpoint
HCI	Human-computer interaction
MG	Module Guide
MIS	Module Interface Specification
PC	Personal computer
PCVR	PC-based virtual reality: FPVR runs on a PC and displays through the headset
PoC	Proof of concept
QA	Quality assurance
SRS	Software Requirements Specification
TA	Teaching assistant
V&V, VnV	Verification and validation
VR	Virtual reality: an immersive presentation viewed through a headset
XR	Extended reality, the umbrella term that includes VR
FR, NFR, SR	Prefixes of SRS functional, nonfunctional, and proposed safety requirement identifiers
T.FR, T.NFR	Prefixes of the functional and nonfunctional system test identifiers in Section 4

2 General Information

2.1 Summary

FPVR is a virtual reality (VR)-based first-person-view (FPV) drone training simulator designed to help beginners develop and improve their drone piloting skills in a safe and accessible environment. Instead of requiring a physical drone, users can operate a simulated drone using VR controls and practise essential flight manoeuvres without the risks of equipment damage or unsafe flying conditions. It is also meant as a tool for experienced Drone fliers to be able to practice in a safe and controlled environment without the pressure of piloting a very expensive piece of equipment.

The system features free-flight practice, progressively challenging obstacle courses, multiple virtual environments, and checkpoints to create a structured learning experience. Users can track their performance through metrics such as completion time, checkpoint progress, and performance ratings, with potential AI-driven feedback to help identify areas for improvement. FPVR also supports registered accounts, saved performance records, attempt history, and local and global leaderboards, allowing users to monitor their progress and compare results with others. Overall, FPVR aims to make learning FPV drone piloting more accessible, engaging, and cost-effective through an immersive virtual training experience.

2.2 Objectives

The primary objective of this Verification and Validation plan is to build confidence in the correctness, reliability, usability, performance, and flight realism of FPVR by verifying and demonstrating that all requirements specified in the Software Requirements Specification (SRS) are satisfied. These qualities are major objectives of the SRS and therefore receive explicit attention throughout verification and validation. Verification and validation efforts will be guided by the Hazard Analysis Document, which will be used to prioritize functionality based on its mission criticality, potential impact, and associated risks. Features that are essential to the safe and reliable operation of the simulator will receive the greatest testing attention to ensure

that critical system behaviours are thoroughly evaluated. Due to limitations in project time and resources, validating the long-term effectiveness of VR-based training, and testing compatibility across different VR headsets will remain out of scope. Additionally, testing the correctness of the underlying Unity VR libraries, frameworks, and headset-specific functionality will not be part of the project, as these are external components assumed to function according to their respective specifications. Verification efforts will instead focus on FPVR's implementation and integration of these components.

2.3 Custom Documents

The project will include two Custom Documents: a Usability Report and a Design Thinking Report. The Usability Report will document a systematic and quantitative evaluation of FPVR's usability by having representative users complete predefined tasks within the simulator. The evaluation will establish testing goals, identify specific user tasks, collect feedback through structured questions and surveys, and measure usability outcomes to identify areas for improvement. The Design Thinking Report will document the human-centred design process used to understand the needs and challenges of beginner FPV drone pilots. This will include conducting user interviews, organizing findings through empathy maps, generating and evaluating potential solutions based on feasibility and desirability, and developing two rounds of prototypes to gather user feedback and refine the proposed design. These documents will help ensure that FPVR's design decisions are informed by user needs and that its usability is evaluated through structured user testing.

2.4 Relevant Documentation

This VnV plan is derived from two project documents: the Software Requirements Specification (SRS) (Cho et al., 2026b) and the Hazard Analysis (Cho et al., 2026a). Together they define what FPVR is supposed to do and which failures would be unacceptable, which is the information needed to choose tests and interpret results.

The SRS (Cho et al., 2026b) is the statement of the software requirements.

It records the goals of FPVR, the functional requirements (for example flight control, checkpoint behaviour, and session flow), and the non-functional requirements (for example performance, usability, and safety-related qualities). System tests in this plan are written against those requirements so that a passing test set is evidence that the implemented product matches the specified behaviour. The SRS also supplies identifiers, assumptions, and environmental constraints that this plan uses for traceability: each system test should map back to one or more requirements.

The Hazard Analysis (Cho et al., 2026a) identifies hazards and risks for FPVR. Through a Failure Mode and Effect Analysis it describes how components can fail, what the effects of those failures would be, and which safety and security requirements mitigate them. Those hazards and the derived safety requirements focus on the cases where incorrect behaviour could injure a user, damage equipment, produce misleading training, or compromise the integrity of the application. Tests inspired by the Hazard Analysis therefore include failure and mitigation scenarios (for example invalid controller input, lost tracking, or unrealistic physics) in addition to the happy-path cases taken from the SRS.

3 Plan

This section describes how FPVR will be verified and validated. It identifies the V&V team, the plans for verifying the SRS, design, V&V plan, and implementation, the automated tools that support them, and the plan for validating FPVR with its intended users. Every in-scope requirement will have a planned verification method, traced in Section 4.3 and reported in the V&V Report.

Table 2 links these activities to the course milestones in SRS P.3 and the evidence each one produces.

Table 2: V&V activities by milestone

Milestone	**V&V activities**	**Evidence**
VnV Plan peer review	Peer and TA review of this plan (Section 3.4)	Logged issues and their resolution
Design Document Rev −1	Review of the MG and MIS, followed by peer review (Section 3.3)	Completed checklist and linked issues
PoC Demonstration	Phases 1–2: deployment, input, safety, and free flight, including the first input-response measurement	Test records and measurement log
Design Documentation	Review of the changed MG and MIS sections (Section 3.3)	Updated checklist and module-to-requirement mapping
Module Implementation	Phase 3: courses, recovery, and timing, including frame-rate and crash-recovery checks; unit and integration suites running in CI	CI test results and coverage report
Revision 0 Demonstration	Phase 4: scoring, review, and local results; supervisor demonstration (Section 3.7)	Demonstration checklist and supervisor feedback
V&V Report	Phase 5: accounts and online results; user studies, mutation exercise, and a full regression run	Study data, test summary, and the V&V Report
Final Demonstration	Phase 6 if included; retest of corrected defects	Retest results

3.1 Verification and Validation Team

The V&V team follows the project roles in SRS P.1, so each area is verified by the member who knows it best:

- **Ibrahim Tariq:** Team Lead and Flight Physics Developer. Coordinates V&V milestones and defect priorities, and leads flight-model, rendering-performance, and flight-realism testing.
- **John Cho:** Team Liaison and Hardware Integration Developer. Leads headset, controller, installation, and interruption-recovery testing, and arranges supervisor and external reviews.
- **Sankalpa Chhetri Dhakal:** Training Environments Lead. Checks every course, obstacle, and recovery position, and leads the interface usability study.
- **Vikram Chandar:** Game State Developer. Leads tests of game states, checkpoint order, crash events, Respawn, Restart Level, and timers.
- **Arjun Sasidaran:** Scoring and Player Data Developer. Leads tests of scoring, ratings, the post-flight review, accounts, result storage, and leaderboards.
- **Dr. Douglas Down (Supervisor):** Reviews scope, flight realism, and acceptance criteria at the fortnightly meetings, and gives feedback on the Revision 0 demonstration.

Review roles rotate on a fixed schedule so that no one verifies their own work and every member reviews every area during the project. The rotation follows the order Ibrahim, John, Sankalpa, Vikram, Arjun:

- **Area reviewer:** Each lead keeps ownership of their area's tests, but the member who reviews them changes at each milestone in Table 2, following the rounds in Table 3. After four milestones, every member has reviewed every other member's area, and the rounds repeat.

- **QA reviewer:** The quality assurance (QA) reviewer for each deliverable is the next member in the order. If that member is the deliverable's main author, the turn passes to the following member.
- **Pull request reviewer:** Every pull request is approved by a member other than its author, normally the current area reviewer for that code.

Assignments are recorded on the GitHub issue for each milestone. Members may swap a turn if the swap is recorded and neither member reviews their own work.

Table 3: Area reviewer rotation

Area lead	Round 1	Round 2	Round 3	Round 4
Ibrahim	John	Sankalpa	Vikram	Arjun
John	Sankalpa	Vikram	Arjun	Ibrahim
Sankalpa	Vikram	Arjun	Ibrahim	John
Vikram	Arjun	Ibrahim	John	Sankalpa
Arjun	Ibrahim	John	Sankalpa	Vikram

3.2 SRS Verification

The SRS will be verified through five activities: a checklist inspection, a supervisor review, peer and TA review, a traceability review, and continuous verification as the requirements change. Every finding is logged as a GitHub issue with an owner and a closing condition. Table 4 summarizes the activities.

3.2.1 Checklist Inspection

Each area lead will prepare the requirements for their area, and a different member will inspect them using the course SRS checklist. The inspection checks that each requirement is unambiguous, testable, complete, and consistent with the rest of the SRS. Two members will also independently calcu-

late expected results from the scoring, rating, and leaderboard rules; different answers show that a rule is ambiguous.

3.2.2 Supervisor Review

A structured review will be held with Dr. Douglas Down at one of the fortnightly meetings. Before the meeting, the team will share a summary of the key requirements, the main training scenarios, and specific questions about high-risk areas. During the meeting, the team will walk through free flight, a course with a crash and Respawn, and a completed result, focusing on flight realism, the reference drone, and whether the score and feedback support beginner practice. Decisions will be recorded in the meeting minutes.

3.2.3 Peer and TA Review

Issues raised by the assigned peer team and feedback from the TA will be checked against the same checklist. Each one will either be accepted with an owner or declined with a recorded reason.

3.2.4 Traceability Review

A non-author will confirm that every requirement traces back to a project goal or hazard and forward to at least one test, and that each safety requirement in the Hazard Analysis is covered by an SRS requirement. Gaps will be logged as issues, and the result is the traceability table in Section 4.3.

3.2.5 Continuous Verification

Requirements will be reviewed again whenever they change, at the weekly team meetings. These reviews will:

- Assess the impact of each requirement change on the design and tests
- Update the affected tests and traceability entries with the requirement

- Confirm at the PoC Demonstration that the performance requirements can be met on the target headset

Table 4: SRS verification checklist

Activity	**Verification tasks**
Checklist inspection	☐ Inspect each area's requirements ☐ Calculate rule examples independently
Supervisor review	☐ Share the review package ☐ Walk through the training scenarios ☐ Record decisions
Peer and TA review	☐ Accept or decline every issue with a reason
Traceability review	☐ Trace requirements to goals, hazards, and tests ☐ Log gaps as issues
Continuous verification	☐ Review requirement changes weekly ☐ Confirm performance requirements at the PoC

3.3 Design Verification

The Module Guide (MG) and Module Interface Specification (MIS) will be verified through walkthroughs, peer review, and supervisor review, first for Design Document Rev −1 (Nov. 16, 2026) and again for the Design Documentation (Jan. 18, 2027). Each area lead will walk a non-author member through their modules' interfaces and state transitions. Classmates will review the design using a checklist during the scheduled peer reviews (Nov. 19, 2026 and Jan. 25, 2027), and their feedback will be tracked in GitHub Issues. The design will use the nine modules of Section 5, so each module's unit tests trace to its requirements.

A review with Dr. Douglas Down will verify three critical aspects of the design: (1) the flight model and its match to the reference drone, (2) the state transitions for checkpoints, crash recovery, and timing, and (3) the handling of lost input and an unavailable online service, which must not block local training. Small prototypes will check uncertain hardware and performance assumptions before the review.

Table 5: Design verification checklist

Focus area	**Verification tasks**
Module structure	☐ Every requirement has an owning module ☐ Modules match the nine modules in Section 5
Interfaces	☐ Inputs, outputs, units, and valid ranges specified ☐ Failure results specified
State transitions	☐ Checkpoint order and crash penalties preserved ☐ Respawn and Restart Level behaviour defined
Failure handling	☐ Lost tracking and input handled ☐ Online failures do not block local training
Testability	☐ Input, time, and services can be controlled in tests

3.4 Verification and Validation Plan Verification

The V&V Plan will be verified through team and peer review. The QA reviewer, the assigned peer team, and the TA will check the plan's coverage and feasibility using a checklist during the VnV Plan peer review (issues due Nov. 5, 2026), and the QA reviewer will repeat the check after major requirement or design changes. Feedback will be tracked through GitHub Issues and addressed at the weekly team meetings.

To confirm that the tests can detect faults, the team will run a mutation exercise once the automated suites exist. Faults will be introduced one at a time on a temporary branch, such as allowing an out-of-order checkpoint, removing a crash penalty, or changing a rating boundary, and each must cause a test to fail. Progress will be measured by requirement coverage, the share of mutations detected, and the number of open review issues.

3.5 Implementation Verification

Implementation verification will use unit, integration, and system tests. The unit tests in Section 5 check each module's rules in isolation using the Unity

Table 6: V&V plan verification checklist

Criteria	**Verification tasks**
Coverage	☐ Every in-scope requirement has a test or inspection ☐ Safety requirements are covered
Test quality	☐ Each test has inputs, an expected result, and a pass criterion ☐ Expected results are derived independently
Feasibility	☐ Equipment, people, and time are available ☐ No procedure is unsafe for participants
Fault detection	☐ Mutation exercise completed
Process	☐ Feedback tracked to closure

Test Framework. Integration tests check the flows between modules, such as input to flight, crash to recovery, and completion to review and storage. The system tests in Section 4 check the complete application on the required hardware configurations in Table 8.

Static verification will use code review and analyzers. Every pull request will be reviewed by a non-author against the Microsoft C# Coding Conventions adopted in the Development Plan, and merged only after approval and a passing CI run. Reviews will focus on:

- Units, bounds, and event ordering in flight and game-state code
- Cleanup of event subscriptions and unintended shared state
- Credential exposure and authorization errors in account and storage code

Performance testing will verify:

- Frame rate on the target headset (NFR-PERF-01)
- Input response time (NFR-PERF-02)

- Reliability over repeated crash and Respawn cycles (NFR-REL-01)

Each failure will become a GitHub issue, and the failing case and affected regression tests will be rerun after the fix.

Table 7: Implementation verification checklist

Category	**Verification tasks**
Functionality	☐ Unit tests for each module ☐ Integration tests for module flows
Quality	☐ Static analysis completed ☐ Code reviews conducted
Performance	☐ Frame-rate measurements ☐ Input-response measurements
Safety	☐ Pause and exit verified ☐ Tracking and input loss handled

3.6 Automated Testing and Verification Tools

Unit Testing Framework: The Unity Test Framework, which is based on NUnit, will run Edit Mode tests for game rules and Play Mode tests for physics and scene behaviour. It was chosen because it is Unity's standard testing tool and runs in the CI pipeline.

Code Coverage Tools and Plan for Summary: The Unity Code Coverage package will report line coverage for each module and overall, excluding third-party and test code. The target is at least 80% line coverage of the game-rule logic, such as checkpoint order, timing, scoring, and leaderboard ranking. Coverage reports will be kept with each CI run to track trends and identify untested code.

Linters and Static Analysis: The C# compiler and Roslyn analyzers will enforce the Microsoft C# Coding Conventions and flag likely defects during development and in the CI build.

Continuous Integration: GitHub Actions will be extended from building the documentation to building the project and running the automated tests on every pull request. A failing run blocks the merge.

Performance Measurement: The Unity Profiler, headset performance metrics, and timestamped logs will measure frame rate and input response on the configurations in Table 8.

Table 8: Required FPVR test configurations

Mode	Execution and display	Flight input
Standalone	FPVR runs on Meta Quest 3S without a connected PC.	Quest motion controllers
PCVR	FPVR runs on the Windows 11 reference PC and displays through Quest 3S.	Xbox-compatible gamepad

3.7 Software Validation

FPVR will be validated with the people it is built for, because requirement-based tests cannot show whether it is a useful trainer. Validation will use:

- **Interface usability study (T.NFR.USAB.02):** About four beginners with little or no FPV experience, split between the two configurations, and the experienced pilots below will complete core tasks without coaching: finding the control guide, starting a course, using Respawn and Restart Level, pausing and exiting, and reading the review. Task completion and questionnaire ratings (Section 6.2) will be recorded, with a target median of at least 4/5. Sankalpa leads this study.
- **Flight realism study:** About two experienced FPV pilots will fly agreed manoeuvres and rate how believable the flight response is, with a target of at least 4/5 on each item. Ibrahim leads this study, and the supervisor helps define the reference drone.
- **Revision 0 demonstration:** At the Revision 0 Demonstration (Feb. 1–11, 2027), Dr. Douglas Down will review whether the functional

requirements in phases 1–4 of SRS P.3 work on both configurations. It succeeds if they can all be demonstrated and no open defect prevents safe use or the core training flow.

Studies will use a clear stationary space, an observer, and a build with verified pause and exit, and participants may stop at any time. Findings will become GitHub issues and will feed the Usability Report and Design Thinking Report.

4 System Tests

System tests are tests that are used to verify that FPVR satisfies the functional and non-functional requirements as specified in the SRS. This section will cover all system tests that will be performed end-to-end on the application.

4.1 Tests for Functional Requirements

The following tests cover the functional requirements in SRS S.2, grouped by the SRS subsection they come from. Each test's output checks every requirement named in its derivation, and Table 9 lists the requirements for each test.

4.1.1 Realistic Flight Simulation (S.2.1)

These tests cover the flight model and crash detection in SRS S.2.1.

First-Person View

1. **T.FR.FLT.01**
   Control: Manual

   Initial State: Free flight on each configuration, with the drone hovering.

Input: Fly forward, climb, and turn using yaw.

Output: During each manoeuvre the view moves and rotates with the drone and stays at the drone's first-person camera position.

Test Case Derivation: FR-FLT-01 requires the view to follow both the position and the orientation of the drone, so the manoeuvres include a translation, a climb, and a rotation.

How test will be performed: A tester performs each manoeuvre in the headset and records whether the view followed the drone.

Independent Control Axes

1. **T.FR.FLT.02**
   Control: Automatic

   Initial State: Drone hovering in an empty test scene.

   Input: Throttle, pitch, roll, and yaw inputs applied one at a time, each at half and at full deflection.

   Output: Each input changes only its matching motion (vertical speed, forward tilt, sideways tilt, or heading), and full deflection produces a larger change than half deflection.

   Test Case Derivation: FR-FLT-02 requires each input to be independent and to produce a corresponding change. Applying one input at a time isolates each axis, and two magnitudes show that the inputs are variable rather than on or off.

   How test will be performed: A Play Mode test applies scripted inputs and records the drone's velocity and orientation after each one.

Gravity Without Thrust

1. **T.FR.FLT.03**
   Control: Automatic

   Initial State: Drone airborne at a fixed height with zero vertical velocity.

Input: Zero throttle for 1.0 s of simulated time.

Output: Vertical velocity decreases throughout the 1.0 s.

Test Case Derivation: FR-FLT-03 states that vertical velocity shall decrease when the drone is airborne and no upward thrust is applied, which is exactly this condition.

How test will be performed: A Play Mode test records the vertical velocity every frame and checks that each value is lower than the one before.

Contact Detection

1. **T.FR.FLT.04**
   Control: Automatic

   Initial State: Drone in flight in a test course containing an obstacle and a ground surface.

   Input: Fly into the obstacle, then in a second run take off and descend onto the ground.

   Output: A contact is detected at the moment of impact in both runs.

   Test Case Derivation: FR-FLT-04 requires contact detection for both course obstacles and ground surfaces, so one run covers each.

   How test will be performed: A Play Mode test flies scripted paths and checks for a contact event at the impact position.

Crash Trigger Rules

1. **T.FR.FLT.05**
   Control: Automatic

   Initial State: New obstacle-course attempt with the drone resting on the starting surface.

   Input: (a) Remain on the ground before takeoff. (b) Take off, then touch the ground. (c) Fly into a designated obstacle.

   Output: (a) No crash. (b) A crash. (c) A crash.

Test Case Derivation: FR-FLT-05 treats initial ground contact differently from ground contact after takeoff and from obstacle contact, so each of the three cases is checked.

How test will be performed: A Play Mode test runs each case from a fresh attempt and checks whether a crash was registered.

Crash Event Handling

1. **T.FR.FLT.06**
   Control: Automatic

   Initial State: Drone airborne in an obstacle course, and in a second run in free flight.

   Input: Hold the drone against an obstacle for 1.0 s, then apply flight input.

   Output: Exactly one crash event is produced, the flight input does not move the drone, and the recovery options for the current mode are shown.

   Test Case Derivation: FR-FLT-06 requires one event per crash, suspended input, and mode-specific recovery. Holding the drone against the obstacle produces repeated contacts, which tests the single-event rule.

   How test will be performed: A Play Mode test counts crash events, applies input after the crash, and checks which recovery menu is displayed in each mode.

4.1.2 Menu Interface (S.2.2, S.2.5)

These tests cover the training modes in SRS S.2.2 and the user interface in S.2.5.

Free-Flight Mode

1. **T.FR.MI.01**
   Control: Manual

   Initial State: Main menu displayed.

   Input: Select free flight and fly for 60 seconds.

   Output: No checkpoints, completion timer, numerical score, or performance rating appear at any time.

   Test Case Derivation: FR-LVL-01 lists four elements that free flight must not have, so the output checks the absence of each one.

   How test will be performed: A tester flies for 60 seconds and records any of the four elements that appear.

Free-Flight Crash Recovery

1. **T.FR.MI.02**
   Control: Manual

   Initial State: Free flight with the drone airborne away from its starting position.

   Input: Crash into an obstacle, then select the free-flight recovery option.

   Output: The drone returns to the free-flight starting position and can be flown again without FPVR restarting.

   Test Case Derivation: FR-LVL-02 requires recovery to the starting position after a crash without restarting the application, so the test starts away from that position and includes a crash.

   How test will be performed: A tester performs the steps and confirms that the session continued and the drone responds to input.

Level Selection

1. **T.FR.MI.03**
   Control: Manual

Initial State: FPVR freshly installed with no saved progress, main menu displayed.

Input: Open the obstacle-course level list and start each listed level.

Output: At least three levels are listed and labelled introductory, intermediate, and advanced, each with a different obstacle or checkpoint arrangement, and every level starts immediately.

Test Case Derivation: FR-LVL-03 requires three labelled levels with distinct arrangements, and FR-LVL-05 requires every level to be available without unlocking. A fresh installation ensures that no earlier progress could have unlocked a level.

How test will be performed: A tester records the level labels and arrangements and starts each level in turn.

Virtual Environments

1. **T.FR.MI.04**
   Control: Manual

   Initial State: Main menu displayed.

   Input: Start an obstacle-course level in each available environment.

   Output: At least two visually distinct environments are available, and each has at least one playable obstacle-course level.

   Test Case Derivation: FR-LVL-04 requires two distinct environments with a playable level in each, so a level is started in every environment.

   How test will be performed: A tester records each environment and the level played in it.

Attempt Initialization

1. **T.FR.MI.05**
   Control: Automatic

   Initial State: A previous attempt on the level ended with 2 checkpoints completed, a total time of 120.0 s, and 3 crashes.

Input: Start a new attempt on the same level.

Output: Checkpoint progress is 0, both timers are 0.0 s, the crash count is 0, and the score state is at its initial value.

Test Case Derivation: FR-LVL-07 requires all four values to be initialized. Starting after an attempt with non-zero values shows that they are reset rather than simply empty.

How test will be performed: A Play Mode test sets up the previous attempt, starts a new one, and reads the four values.

Return to Main Menu

1. **T.FR.MI.06**
   Control: Manual

   Initial State: Active flight in free flight, and in a second run in an obstacle course.

   Input: Select the option to return to the main menu.

   Output: The main menu is displayed in both runs without FPVR restarting.

   Test Case Derivation: FR-LVL-06 covers both training modes, so the test is run from each.

   How test will be performed: A tester performs the steps and confirms that the session continued.

Menu Interactions in the Headset

1. **T.FR.MI.07**
   Control: Manual

   Initial State: FPVR running in the target headset with the main menu displayed.

   Input: Using only the headset and controller, choose a training mode, select a level, start flight, crash and recover, and return to the main menu.

Output: Each action is available through a menu interaction, and the flight view and every menu are shown in the headset.

Test Case Derivation: FR-UI-05 lists these five interactions, and FR-UI-01 requires the flight view and menus to appear in the headset, so the sequence covers each one.

How test will be performed: A tester completes the sequence on each configuration and records any action that was not available.

Control Guide Before Flight

1. **T.FR.MI.08**
   Control: Manual

   Initial State: FPVR newly launched, once on each configuration.

   Input: Proceed from the main menu to the first flight.

   Output: Before flight begins, a control guide identifies the throttle, pitch, roll, and yaw inputs for the controller in use.

   Test Case Derivation: FR-UI-04 requires the guide for each supported configuration before flight, and the assigned inputs differ between controllers, so each configuration is checked.

   How test will be performed: A tester compares the guide shown against the controller mapping for that configuration.

Execution Modes

1. **T.FR.MI.09**
   Control: Manual

   Initial State: Release build installed on Quest 3S and on the reference PCVR computer.

   Input: Start FPVR standalone on the headset, then start it on the reference computer with the headset connected.

   Output: FPVR runs and can start a flight in both modes.

Test Case Derivation: FR-UI-02 requires both standalone and PCVR execution, so a flight is started in each mode.

How test will be performed: A tester starts each mode and records whether a flight could be started.

Controller Configurations

1. **T.FR.MI.10**
   Control: Manual

   Initial State: FPVR running in free flight, once in each mode.

   Input: Use the throttle, pitch, roll, and yaw controls with the Quest motion controllers in standalone mode and with the standard gamepad in PCVR mode.

   Output: Each configuration provides all four controls, and each one moves the drone.

   Test Case Derivation: FR-UI-03 requires a configuration for each controller and mode pair with all four controls, so every control is used in both configurations.

   How test will be performed: A tester records the result for each control in each configuration.

Seated and Standing Operation

1. **T.FR.MI.11**
   Control: Manual

   Initial State: FPVR running, with the tester in a marked stationary area.

   Input: Fly a course once seated and once standing.

   Output: The course can be flown in both positions without walking to control the drone.

   Test Case Derivation: FR-UI-06 requires seated and standing operation without walking, so the same course is flown in each position.

How test will be performed: A tester flies the course in each position and records any time they had to walk.

4.1.3 Checkpoint and Crash Recovery (S.2.3)

These tests cover checkpoint order, level completion, and the two recovery options in SRS S.2.3.

Checkpoint Definitions

1. **T.FR.CHK.01**
   Control: Automatic

   Initial State: All released obstacle-course levels.

   Input: Load each level's course definition.

   Output: Every level has an ordered list of at least two checkpoints followed by a finish location.

   Test Case Derivation: FR-CHK-01 requires this structure for each level, so every level is checked.

   How test will be performed: An Edit Mode test loads each level definition and checks the checkpoint list and finish.

Ordered Checkpoint Completion

1. **T.FR.CHK.02**
   Control: Automatic

   Initial State: Level with checkpoints A, B, and C, none completed.

   Input: Pass through B, then A, then B.

   Output: After the first B: 0 completed. After A: 1 completed. After the second B: 2 completed.

   Test Case Derivation: Passing B first skips A, so FR-CHK-03 requires no progress. The later passes follow the order, so FR-CHK-02 requires each to count.

How test will be performed: A Play Mode test flies the drone through the checkpoints and reads progress after each pass.

Level Completion

1. **T.FR.CHK.03**
   Control: Automatic

   Initial State: Level with checkpoints A, B, and C, with A and B completed.

   Input: Reach the finish, then complete C and reach the finish again.

   Output: The level is not complete after the first finish and is complete after the second.

   Test Case Derivation: FR-CHK-08 requires every checkpoint in order before completion, so the first finish with C missing must not count.

   How test will be performed: A Play Mode test reads the completion state after each finish.

Recovery Choices

1. **T.FR.CHK.04**
   Control: Manual

   Initial State: Active obstacle-course attempt.

   Input: Crash into an obstacle.

   Output: The recovery menu shows the choices Respawn and Restart Level.

   Test Case Derivation: FR-CHK-04 requires both named choices after a crash in an obstacle course.

   How test will be performed: A tester crashes on each configuration and records the choices shown.

Respawn

1. **T.FR.CHK.05**
   Control: Automatic

   Initial State: (a) Crash before any checkpoint. (b) Crash with checkpoint A completed, a total time of 40.0 s, and a crash count of 1.

   Input: Select Respawn in each case.

   Output: (a) The drone is at the level start. (b) The drone is at checkpoint A, A is still completed, the total time is 40.0 s, and the crash count is 1.

   Test Case Derivation: FR-CHK-05 sets the respawn position with and without a completed checkpoint, so both cases are covered. FR-CHK-06 requires progress, total time, and crash count to be kept, so case (b) uses non-zero values for each.

   How test will be performed: A Play Mode test sets up each case, selects Respawn, and reads the drone position and attempt values.

Restart Level

1. **T.FR.CHK.06**
   Control: Automatic

   Initial State: Crash with checkpoint A completed, a total time of 40.0 s, a section time of 12.0 s, and a crash count of 1.

   Input: Select Restart Level.

   Output: The drone is at the level start, no checkpoints are completed, both timers are 0.0 s, the crash count is 0, and the score state is reset.

   Test Case Derivation: FR-CHK-07 lists the values that Restart Level resets, and each starts non-zero so that the reset is visible.

   How test will be performed: A Play Mode test sets up the state, selects Restart Level, and reads each value.

4.1.4 Timing and Performance Evaluation (S.2.4)

These tests cover the timers, crash count, score, and rating in SRS S.2.4. Simulated time is used so that the expected values are exact.

Total Timer

1. **T.FR.PER.01**
   Control: Automatic

   Initial State: New obstacle-course attempt using simulated time.

   Input: Fly 10.0 s, crash, select Respawn, fly 20.0 s to the finish, then wait 5.0 s.

   Output: The total timer reads 30.0 s at completion and still reads 30.0 s after the wait, and the score and review use 30.0 s.

   Test Case Derivation: FR-PER-01 counts active flight time (10.0 + 20.0 = 30.0 s), FR-PER-02 keeps the 10.0 s from the segment that ended in a crash, and FR-PER-10 stops the timer at completion and uses its final value.

   How test will be performed: A Play Mode test advances simulated time and reads the timer, score input, and review value.

Section Timer

1. **T.FR.PER.02**
   Control: Automatic

   Initial State: New obstacle-course attempt using simulated time.

   Input: Fly 8.0 s and complete checkpoint A, fly 5.0 s, crash, and select Respawn.

   Output: The section timer reads 8.0 s before A, 0.0 s after A, 5.0 s at the crash, and 0.0 s after Respawn.

   Test Case Derivation: FR-PER-03 times the current segment, and FR-PER-04 resets it at checkpoint completion and at Respawn, so the timer is read before and after each event.

How test will be performed: A Play Mode test reads the section timer after each step.

Timers After Recovery

1. **T.FR.PER.03**
   Control: Automatic

   Initial State: Crash with a total time of 40.0 s.

   Input: Select Respawn and read the timers, then crash again and select Restart Level.

   Output: After Respawn the total time is 40.0 s. After Restart Level both timers are 0.0 s.

   Test Case Derivation: FR-PER-05 keeps the total time on Respawn and resets both timers on Restart Level.

   How test will be performed: A Play Mode test reads both timers after each choice.

Timers Paused in Recovery Menu

1. **T.FR.PER.04**
   Control: Automatic

   Initial State: Active attempt with a total time of 30.0 s and a section time of 8.0 s.

   Input: Crash, leave the recovery menu open for 5.0 s of simulated time, select Respawn, then fly 2.0 s.

   Output: Both timers are unchanged while the menu is open, and the total time is 32.0 s after the flight.

   Test Case Derivation: FR-PER-06 pauses both timers during the recovery menu and adds no menu time, so the 5.0 s must not appear in the total.

   How test will be performed: A Play Mode test reads the timers before, during, and after the menu.

Crash Count

1. **T.FR.PER.05**
   Control: Automatic

   Initial State: New obstacle-course attempt.

   Input: Three separate crashes with Respawn after each, one of them with repeated contact against the same obstacle.

   Output: The crash count is 3.

   Test Case Derivation: FR-PER-07 adds one per registered crash, so repeated contact within one crash must not add extra counts.

   How test will be performed: A Play Mode test performs the crashes and reads the count.

Score Calculation

1. **T.FR.PER.06**
   Control: Automatic

   Initial State: Completed attempts with known values.

   Input: (a) $T = 95.7$ s and $C = 2$. (b) $T = 950.0$ s and $C = 1$.

   Output: (a) A score of 705. (b) A score of 0.

   Test Case Derivation: By FR-PER-08, (a) gives $1000 - 95 - 200 = 705$ and (b) gives $1000 - 950 - 100 = -50$, which the maximum with 0 raises to 0.

   How test will be performed: An automated test completes each attempt and compares the displayed score.

Rating Bands

1. **T.FR.PER.07**
   Control: Automatic

   Initial State: Rating logic for completed attempts.

Input: Scores of 0, 399, 400, 599, 600, 799, 800, and 1000.

Output: Needs Improvement for 0 and 399, Bronze for 400 and 599, Silver for 600 and 799, and Gold for 800 and 1000.

Test Case Derivation: FR-PER-09 defines four bands, and these inputs are the boundaries of each band.

How test will be performed: An automated test assigns each score and compares the rating.

4.1.5 Player Accounts and Authentication (S.2.6)

These tests cover registration, sign-in, and sign-out in SRS S.2.6. They use test accounts on the online service.

Account Registration

1. **T.FR.ACC.01**
   Control: Manual

   Initial State: Network connectivity and an unused username.

   Input: Register with the unused username and a 12-character password.

   Output: The account is created and can be used to sign in.

   Test Case Derivation: FR-ACC-01 requires registration with a unique username and password, and 12 characters is the shortest password FR-ACC-02 allows.

   How test will be performed: A tester registers on a test account and then signs in with it.

Registration Rejection

1. **T.FR.ACC.02**
   Control: Manual

   Initial State: Network connectivity and one existing account.

Input: Register with the existing username, then with a new username and an 11-character password.

Output: Both registrations are rejected, each with a message explaining the reason.

Test Case Derivation: FR-ACC-02 rejects duplicate usernames and passwords under 12 characters, so 11 characters is the boundary case.

How test will be performed: A tester attempts each registration and records the message shown.

Sign-In

1. **T.FR.ACC.03**
   Control: Manual

   Initial State: One registered account.

   Input: Sign in with valid credentials, then with a wrong password, then with an unknown username.

   Output: The valid sign-in succeeds, and both invalid attempts show the same message.

   Test Case Derivation: FR-ACC-03 requires valid sign-in to succeed and invalid attempts not to reveal which field was wrong, so both invalid cases must give the same message.

   How test will be performed: A tester performs each sign-in and records the result and message.

Sign-Out

1. **T.FR.ACC.04**
   Control: Manual

   Initial State: Signed-in player.

   Input: Sign out, then try to submit a result to the global leaderboard.

   Output: The session ends, and the submission asks the player to sign in.

Test Case Derivation: FR-ACC-04 requires sign-out to end the session, which is shown by authentication being required again.

How test will be performed: A tester performs the steps and records the session state.

Practice Without an Account

1. **T.FR.ACC.05**
   Control: Manual

   Initial State: FPVR with no account signed in.

   Input: Use free flight, complete an obstacle course, then try to submit the result to the global leaderboard.

   Output: Both training modes work, and the submission is not sent until the player signs in.

   Test Case Derivation: FR-ACC-05 allows practice without an account but requires authentication for global submission, so both parts are checked.

   How test will be performed: A tester performs the steps and confirms on the service that nothing was submitted.

4.1.6 Result Storage and Leaderboards (S.2.7)

These tests cover result storage, synchronization, and leaderboards in SRS S.2.7.

Local Result Record

1. **T.FR.DAT.01**
   Control: Automatic

   Initial State: Empty local storage.

   Input: Complete an obstacle-course attempt with known values.

Output: A local record is saved with the level identifier, completion timestamp, total time, crash count, score, rating, and local player identifier, each matching the attempt.

Test Case Derivation: FR-DAT-01 lists these seven fields, so each is compared with the attempt.

How test will be performed: An automated test completes the attempt and reads the saved record.

Online Result Submission

1. **T.FR.DAT.02**
   Control: Automatic

   Initial State: Signed-in player with network connectivity.

   Input: Complete an obstacle-course attempt.

   Output: The result is stored by the online service and linked to the player's account.

   Test Case Derivation: FR-DAT-02 requires submission and association with the account, so the service record is checked for both.

   How test will be performed: An automated test completes the attempt and queries the test service.

Failed Submission and Retry

1. **T.FR.DAT.03**
   Control: Automatic

   Initial State: Signed-in player with the online service unavailable.

   Input: Complete an attempt, restore the service, and retry synchronization.

   Output: Before the retry, the result is kept locally and marked as local only. After the retry, it is on the service and marked as synchronized.

   Test Case Derivation: FR-DAT-03 requires the result to be kept and retried, and FR-DAT-10 requires the two states to be distinguished, which this sequence shows.

How test will be performed: An automated test switches the test service off and on and reads the result's state.

Local Leaderboard

1. **T.FR.DAT.04**
   Control: Manual

   Initial State: Twelve local results for one level on the current installation.

   Input: Open the local leaderboard for that level.

   Output: At least the top ten results are shown, each with rank, player display name, score, completion time, and crash count.

   Test Case Derivation: FR-DAT-04 requires a local leaderboard per level, and FR-DAT-07 requires at least ten entries with these fields. Twelve results make it possible to check the ten-entry minimum.

   How test will be performed: A tester opens the leaderboard and checks each field.

Global Leaderboard

1. **T.FR.DAT.05**
   Control: Automatic

   Initial State: Online service with accepted results for one level, and one local result that was not synchronized.

   Input: Open the global leaderboard for that level.

   Output: The accepted results are shown, and the unsynchronized local result is not.

   Test Case Derivation: FR-DAT-05 uses only results accepted by the online service, so an unsynchronized result must be absent.

   How test will be performed: An automated test compares the leaderboard with the service records.

Leaderboard Ranking

1. **T.FR.DAT.06**
   Control: Automatic

   Initial State: Results for one level: A (705, 195.0 s, 10:05), B (705, 95.0 s, 10:10), C (705, 95.0 s, 10:00), and D (800, 150.0 s, 10:20).

   Input: Rank the results on the local and global leaderboards.

   Output: The order is D, C, B, A on both leaderboards.

   Test Case Derivation: FR-DAT-06 ranks by score, then shorter time, then earlier timestamp. D has the highest score, A has the longest time, and C and B are separated by timestamp.

   How test will be performed: An automated test loads the results and reads both rankings.

Online Leaderboard Unavailable

1. **T.FR.DAT.07**
   Control: Manual

   Initial State: Online service unavailable.

   Input: Open the global leaderboard, then the local leaderboard, then a training mode.

   Output: An availability message is shown, and the local leaderboard and training modes still work.

   Test Case Derivation: FR-DAT-08 requires both the message and continued local use.

   How test will be performed: A tester blocks network access and follows the steps.

Results on Another Installation

1. **T.FR.DAT.08**
   Control: Manual

Initial State: Registered player with a synchronized result.

Input: Sign in on a second supported installation and open the player's results.

Output: The synchronized result is shown with the same values.

Test Case Derivation: FR-DAT-09 requires retrieval after signing in on another installation.

How test will be performed: A tester compares the values on both installations.

4.1.7 Post-Flight Review and Intelligent Feedback (S.2.8, S.2.9)

These tests cover the post-flight review in SRS S.2.8 and the optional intelligent feedback in S.2.9.

Review Contents

1. **T.FR.REV.01**
   Control: Manual

   Initial State: Introductory level completed with $T = 95.7$ s, 3 of 3 checkpoints, and $C = 2$.

   Input: Finish the level.

   Output: The review appears immediately and shows the introductory level, 95.7 s, 3 of 3 checkpoints, 2 crashes, a score of 705, and a Silver rating.

   Test Case Derivation: FR-REV-01 requires the review right after completion, and FR-REV-02 lists these six values. By FR-PER-08 and FR-PER-09, $T = 95.7$ and $C = 2$ give 705 and Silver.

   How test will be performed: A tester completes the level with a scripted run and compares the review values.

Review Navigation

1. **T.FR.REV.02**
   Control: Manual

   Initial State: Post-flight review displayed.

   Input: Select retry, then from a new review select another level, then from a new review return to the main menu.

   Output: Each choice opens the expected screen without FPVR restarting.

   Test Case Derivation: FR-REV-03 lists these three choices, so each is used once.

   How test will be performed: A tester performs each choice and records the screen reached.

Review Without Services

1. **T.FR.REV.03**
   Control: Manual

   Initial State: Network and intelligent feedback service unavailable.

   Input: Complete an obstacle-course level.

   Output: The standard review is displayed with all of its values.

   Test Case Derivation: FR-REV-04 and FR-AI-03 require the standard review even when these services are unavailable.

   How test will be performed: A tester blocks network access and completes the level.

Prior Session Results

1. **T.FR.REV.04**
   Control: Manual

   Initial State: Registered player with completed results from an earlier session.

   Input: Restart FPVR, sign in, and open the stored results.

Output: The earlier results are shown with every field in FR-DAT-01.

Test Case Derivation: FR-REV-05 requires stored results from prior sessions with at least the FR-DAT-01 fields.

How test will be performed: A tester compares the shown results with the earlier records.

Intelligent Feedback

1. **T.FR.AI.01**
   Control: Manual

   Initial State: Build with intelligent feedback included and the feedback service available.

   Input: Complete an obstacle-course level with 2 crashes.

   Output: Without any extra request, the review shows a natural-language observation that refers to the attempt's time, checkpoints, or crashes.

   Test Case Derivation: FR-AI-01 requires the observation to be based on the attempt data, and FR-AI-02 requires it to appear automatically. This test applies only if intelligent feedback is included.

   How test will be performed: A tester compares the observation with the attempt's recorded values.

4.2 Tests for Nonfunctional Requirements

This section describes the system tests for the nonfunctional requirements in SRS S.2, together with the safety requirements proposed in the SRS. Each subsection covers one quality area. Some tests produce measurements or survey results rather than a simple pass or fail, and those results are reported in the V&V Report.

4.2.1 Installability and Compatibility

This area checks that FPVR installs and runs in both deployment modes with the required controllers.

Standalone Installation

1. **T.NFR.INST.01**
   Type: Nonfunctional, Dynamic, Manual

   Initial State: Meta Quest 3S with no FPVR build installed and no computer connected.

   Input/Condition: Install the release build on the headset and launch FPVR.

   Output/Result: FPVR installs and launches to the main menu without a connected computer. (NFR-INST-01)

   How test will be performed: A tester installs the tagged release build and records the build, headset software version, and any errors.

PCVR Launch

1. **T.NFR.INST.02**
   Type: Nonfunctional, Dynamic, Manual

   Initial State: Reference Windows 11 PC with the release build installed and Quest 3S connected through a supported PCVR link.

   Input/Condition: Launch FPVR from the reference PC.

   Output/Result: FPVR launches in PCVR mode and displays the main menu in the headset. (NFR-INST-02)

   How test will be performed: A tester launches the build and records the PC configuration and PCVR link method.

Controller Compatibility

1. **T.NFR.COMP.01**
   Type: Nonfunctional, Dynamic, Manual

   Initial State: FPVR running in free flight, once in each configuration in Table 8.

   Input/Condition: Apply full throttle, pitch, roll, and yaw input one axis at a time with the Quest motion controllers in standalone mode and the Xbox-compatible gamepad in PCVR mode.

   Output/Result: Each input moves only the matching flight axis in both configurations. (NFR-COMP-01)

   How test will be performed: A tester records the response of each axis for each configuration in a checklist.

4.2.2 Performance

This area checks that FPVR stays responsive on the target hardware. These tests produce measurements that are reported in tables as well as a pass or fail result.

Rendering Frame Rate

1. **T.NFR.PERF.01**
   Type: Nonfunctional, Dynamic, Automatic

   Initial State: Standalone build running on Quest 3S in each of the two required environments.

   Input/Condition: 120 seconds of continuous normal flight in each environment.

   Output/Result: The average rendered frame rate is at least 72 FPS in each environment. Frame-time spikes are also reported. (NFR-PERF-01)

   How test will be performed: Frame times are recorded with the Unity Profiler and headset performance metrics, and the average, minimum, and largest spikes are reported in a table for each environment.

Input Response Time

1. **T.NFR.PERF.02**
   Type: Nonfunctional, Dynamic, Automatic

   Initial State: Drone in flight in the reference test environment, in each configuration.

   Input/Condition: 100 flight-control input events.

   Output/Result: At least 95 of the 100 events reach their first simulated state update within 100 ms. The median, 95th percentile, and maximum are reported. (NFR-PERF-02)

   How test will be performed: A common clock timestamps each received input event and its first corresponding state update, and the results are reported separately for each configuration.

4.2.3 Reliability

This area checks that the crash recovery workflow, which users repeat often, stays stable.

Crash Recovery Stress

1. **T.NFR.REL.01**
   Type: Nonfunctional, Dynamic, Automatic

   Initial State: Obstacle-course attempt with at least one checkpoint completed, in each required environment.

   Input/Condition: 20 consecutive crash-and-respawn cycles.

   Output/Result: FPVR does not terminate, and checkpoint progress stays the same after every cycle. (NFR-REL-01)

   How test will be performed: A Play Mode test drives the crashes and respawns and checks checkpoint progress after each cycle.

4.2.4 Usability

This area checks that the control guide is easy to find and that beginners can learn the interface. The study follows Section 3.7, and its questions are in Section 6.2.

Control Guide Access

1. **T.NFR.USAB.01**
   Type: Nonfunctional, Dynamic, Manual

   Initial State: FPVR newly launched on each configuration.

   Input/Condition: Open the control guide from the main menu, then start a first flight.

   Output/Result: The control guide is reachable from the main menu and is shown before the first flight of the session. (NFR-USAB-01)

   How test will be performed: A tester follows the steps on each configuration and records whether each condition is met.

Interface Usability Study

1. **T.NFR.USAB.02**
   Type: Nonfunctional, Dynamic, Manual

   Initial State: Build with verified pause and exit, about four beginners with little or no FPV experience split between the two configurations, and the experienced pilots from the flight realism study in Section 3.7.

   Input/Condition: Participants complete the core tasks without coaching: find the control guide, start a course, use Respawn and Restart Level, pause and exit, and read the review. They then complete the questionnaire in Section 6.2.

   Output/Result: At least three of four beginners complete each task without help, all participants pause and exit successfully, and each questionnaire item has a median of at least 4/5. (NFR-USAB-01, NFR-SAFE-01)

How test will be performed: An observer records task completion, time, errors, and assistance. Results are reported by participant group, and findings become GitHub issues.

4.2.5 Safety

This area checks that users can stop safely and that FPVR handles failures without putting the user at risk. SR-01 to SR-08 are proposed in the SRS, so their tests apply only if the requirements are approved.

Pause and Exit

1. **T.NFR.SAFE.01**
   Type: Nonfunctional, Dynamic, Manual

   Initial State: Active flight in free flight and in an obstacle course, on each configuration.

   Input/Condition: Open the in-application menu, pause, then exit the activity.

   Output/Result: The user can pause and leave without removing the headset, and flight and both timers stop while paused. (NFR-SAFE-01, SR-03)

   How test will be performed: A tester performs the steps and records the timer values before and after the pause.

Stationary Use

1. **T.NFR.SAFE.02**
   Type: Nonfunctional, Dynamic, Manual

   Initial State: FPVR running on each configuration.

   Input/Condition: Complete flight, menu, and recovery actions once seated and once standing in one place.

   Output/Result: Every action can be completed without walking, and no menu requires reaching the floor. (NFR-SAFE-02, SR-03)

How test will be performed: A tester performs the actions in a marked stationary area and records any action that required moving.

Safety Notice

1. **T.NFR.SAFE.03**
   Type: Nonfunctional, Dynamic, Manual

   Initial State: FPVR newly installed.

   Input/Condition: Launch FPVR and proceed to the first flight.

   Output/Result: The safety, age, space, posture, and discomfort information is shown before the first flight, with no unsupported medical or room claims. (SR-01)

   How test will be performed: A tester compares the notice against the SR-01 checklist.

Loss of Tracking or Input

1. **T.NFR.SAFE.04**
   Type: Nonfunctional, Dynamic, Manual

   Initial State: Active obstacle-course flight.

   Input/Condition: Simulate loss of tracking, input focus, and controller connection, one at a time.

   Output/Result: Flight and both timers stop, stale controls are discarded, and resuming requires available input, neutral controls, and confirmation. (SR-02)

   How test will be performed: A tester causes each loss, records the timer values, and attempts to resume without neutral controls.

Sustained Low Frame Rate

1. **T.NFR.SAFE.05**
   Type: Nonfunctional, Dynamic, Automatic

Initial State: Active flight with a test hook that lowers the frame rate.

Input/Condition: Hold the frame rate below the target for longer than the failure duration in the SRS.

Output/Result: FPVR pauses, explains why, and resumes only after confirmation. (SR-04)

How test will be performed: A Play Mode test induces the drop and checks the pause state and message. No participant is exposed to the low frame rate.

Invalid Input and Configuration

1. **T.NFR.SAFE.06**
   Type: Nonfunctional, Dynamic, Automatic

   Initial State: FPVR with test fixtures for input and level data.

   Input/Condition: Non-finite and out-of-range inputs, an invalid checkpoint order, and an intersecting recovery position.

   Output/Result: Invalid configuration is rejected before flight, run-time faults enter a recoverable error state, and progress, scores, and controls are unaffected. (SR-05)

   How test will be performed: Automated tests inject each fault and compare the state before and after.

4.2.6 Security and Privacy

This area checks that account data is protected and that FPVR sends and keeps only the data it needs.

Credential Protection

1. **T.NFR.SEC.01**
   Type: Nonfunctional, Static and Dynamic, Automatic

Initial State: Test account on the online service and FPVR with local storage cleared.

Input/Condition: Register and sign in while capturing network traffic, then inspect local storage and the service's stored account record.

Output/Result: Credentials are sent only over encrypted connections, and no password appears in plaintext on the device or the service. (NFR-SEC-01)

How test will be performed: Network traffic is captured with a proxy, and storage is searched for the known test password. The account code is also reviewed for credential handling.

Access Control

1. **T.NFR.SEC.02**
   Type: Nonfunctional, Dynamic, Automatic

   Initial State: Two test accounts with stored results.

   Input/Condition: Without signing in, request the other account's non-public data and submit a result using its identifier.

   Output/Result: Both requests are rejected, and no data is returned or changed. (NFR-SEC-02)

   How test will be performed: Automated requests are sent to the service, and the stored records are compared before and after.

Data Minimization

1. **T.NFR.SEC.03**
   Type: Nonfunctional, Static and Dynamic, Manual

   Initial State: FPVR with network capture enabled.

   Input/Condition: Play and review several attempts while signed out, then sign in and submit a result.

   Output/Result: No attempt data is sent while signed out, only the result record fields (level, timestamp, total time, crash count, score,

rating, and player) are sent after sign-in, and only result records remain in local storage. (SR-06, SR-07)

How test will be performed: Network traffic and local storage are inspected after each step.

Feedback Isolation

1. **T.NFR.SEC.04**
   Type: Nonfunctional, Static and Dynamic, Automatic

   Initial State: Build with intelligent feedback included.

   Input/Condition: Review the feedback provider's privacy and retention terms, then complete attempts with feedback available, timed out, and declined.

   Output/Result: Feedback is labelled advisory, cannot change checkpoint, timer, crash, score, or rating data, and the standard review stays intact in every case. (SR-08)

   How test will be performed: The provider review is recorded, and automated tests compare result data before and after feedback.

4.2.7 Data Persistence and Offline Availability

This area checks that results are kept across sessions and that core training works without the online service.

Result Persistence

1. **T.NFR.DAT.01**
   Type: Nonfunctional, Dynamic, Manual

   Initial State: Signed-in player with a result confirmed as synchronized.

   Input/Condition: Sign out, restart FPVR, and sign in again.

   Output/Result: The synchronized result is still retrievable with the same values. (NFR-DAT-01)

How test will be performed: A tester records the result values before and after the steps.

Offline Availability

1. **T.NFR.OFF.01**
   Type: Nonfunctional, Dynamic, Manual

   Initial State: FPVR running with the online service unreachable.

   Input/Condition: Use free flight, play a course, crash and recover, save the result, and open the review.

   Output/Result: Every listed action works, and only online features report that the service is unavailable. (NFR-OFF-01)

   How test will be performed: A tester blocks network access and follows the steps on each configuration.

4.3 Traceability Between Test Cases and Requirements

This section links each system test to the requirements it verifies. Table 9 covers the functional requirements, and Table 10 covers the nonfunctional requirements and the proposed safety requirements. The unit tests in Section 5 give additional evidence for the rules behind many of these requirements.

Table 9: Traceability between functional requirements and system tests

Test	Requirements	Test	Requirements
T.FR.FLT.01	FR-FLT-01	T.FR.PER.02	FR-PER-03, FR-PER-04
T.FR.FLT.02	FR-FLT-02	T.FR.PER.03	FR-PER-05
T.FR.FLT.03	FR-FLT-03	T.FR.PER.04	FR-PER-06
T.FR.FLT.04	FR-FLT-04	T.FR.PER.05	FR-PER-07
T.FR.FLT.05	FR-FLT-05	T.FR.PER.06	FR-PER-08
T.FR.FLT.06	FR-FLT-06	T.FR.PER.07	FR-PER-09
T.FR.MI.01	FR-LVL-01	T.FR.ACC.01	FR-ACC-01
T.FR.MI.02	FR-LVL-02	T.FR.ACC.02	FR-ACC-02
T.FR.MI.03	FR-LVL-03, FR-LVL-05	T.FR.ACC.03	FR-ACC-03
T.FR.MI.04	FR-LVL-04	T.FR.ACC.04	FR-ACC-04
T.FR.MI.05	FR-LVL-07	T.FR.ACC.05	FR-ACC-05
T.FR.MI.06	FR-LVL-06	T.FR.DAT.01	FR-DAT-01
T.FR.MI.07	FR-UI-01, FR-UI-05	T.FR.DAT.02	FR-DAT-02
T.FR.MI.08	FR-UI-04	T.FR.DAT.03	FR-DAT-03, FR-DAT-10
T.FR.MI.09	FR-UI-02	T.FR.DAT.04	FR-DAT-04, FR-DAT-07
T.FR.MI.10	FR-UI-03	T.FR.DAT.05	FR-DAT-05
T.FR.MI.11	FR-UI-06	T.FR.DAT.06	FR-DAT-06
T.FR.CHK.01	FR-CHK-01	T.FR.DAT.07	FR-DAT-08
T.FR.CHK.02	FR-CHK-02, FR-CHK-03	T.FR.DAT.08	FR-DAT-09
T.FR.CHK.03	FR-CHK-08	T.FR.REV.01	FR-REV-01, FR-REV-02
T.FR.CHK.04	FR-CHK-04	T.FR.REV.02	FR-REV-03
T.FR.CHK.05	FR-CHK-05, FR-CHK-06	T.FR.REV.03	FR-REV-04, FR-AI-03
T.FR.CHK.06	FR-CHK-07	T.FR.REV.04	FR-REV-05
T.FR.PER.01	FR-PER-01, FR-PER-02, FR-PER-10	T.FR.AI.01	FR-AI-01, FR-AI-02

Table 10: Traceability between nonfunctional requirements and system tests

System test case ID	Requirements verified
T.NFR.INST.01	NFR-INST-01
T.NFR.INST.02	NFR-INST-02
T.NFR.COMP.01	NFR-COMP-01
T.NFR.PERF.01	NFR-PERF-01
T.NFR.PERF.02	NFR-PERF-02
T.NFR.REL.01	NFR-REL-01
T.NFR.USAB.01	NFR-USAB-01
T.NFR.USAB.02	NFR-USAB-01, NFR-SAFE-01
T.NFR.SAFE.01	NFR-SAFE-01, SR-03
T.NFR.SAFE.02	NFR-SAFE-02, SR-03
T.NFR.SAFE.03	SR-01
T.NFR.SAFE.04	SR-02
T.NFR.SAFE.05	SR-04
T.NFR.SAFE.06	SR-05
T.NFR.SEC.01	NFR-SEC-01
T.NFR.SEC.02	NFR-SEC-02
T.NFR.SEC.03	SR-06, SR-07
T.NFR.SEC.04	SR-08
T.NFR.DAT.01	NFR-DAT-01
T.NFR.OFF.01	NFR-OFF-01

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access

program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

Most nonfunctional requirements depend on the full system or target hardware, so they are tested in section 4.2. For example, rendering performance (NFR-PERF-01) must be measured on the headset. The tests below give earlier evidence at the module level. The input response test is the one exception to isolation, because NFR-PERF-02 measures the time from an input event to the flight update.

5.3.1 VR Interface and Input

1. `ProcessInput_HundredEvents_MeetsResponseLimit`

Type: Nonfunctional, Dynamic, Automatic

Initial State: Active drone in an empty Play Mode test scene on the reference PCVR computer.

Input: 100 simulated flight input events.

Expected Output: At least 95% of events are within the NFR-PERF-02 limit. Median, 95th percentile, and maximum response times are recorded in a table.

Requirement Traceability: Supports NFR-PERF-02 (input response).

5.3.2 Checkpoint Progress and Recovery

1. `RespawnCycle_TwentyCrashes_KeepsProgress`

   Type: Nonfunctional, Dynamic, Automatic

   Initial State: Attempt with two checkpoints complete.

   Input: 20 crash-and-respawn cycles.

   Expected Output: Both checkpoints remain complete, and no exceptions occur.

   Requirement Traceability: Supports NFR-REL-01 (crash-recovery reliability).

5.3.3 Accounts and Online Service

1. `Register_ValidPassword_NoPlaintextStored`

   Type: Nonfunctional, Dynamic, Automatic

   Initial State: Empty local storage and a fake online service.

   Input: Registration with a known test password.

   Expected Output: The password does not appear in any locally saved data.

   Requirement Traceability: Supports NFR-SEC-01 (credential protection).

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

John Cho, Ibrahim Tariq, Vikram Chandar, Sankalpa Chhetri Dhakal, and Arjun Sasidaran. Hazard analysis for fpvr. https://github.com/SoftwareMandems/FPVR/blob/main/docs/HazardAnalysis/HazardAnalysis.pdf, 2026a.

John Cho, Ibrahim Tariq, Vikram Chandar, Sankalpa Chhetri Dhakal, and Arjun Sasidaran. Software requirements specification for fpvr. https://github.com/SoftwareMandems/FPVR/blob/main/docs/SRS/SRS.pdf, 2026b.

6 Appendix

6.1 Symbolic Parameters

The system tests defined in Section 4 use symbolic constants to specify test conditions, acceptance thresholds, and expected results. Defining these parameters in one location ensures consistency across verification activities and allows their values to be updated without introducing conflicting test criteria.

The constants in Table 11 are derived from the Software Requirements Specification (SRS) and the Hazard Analysis. The SRS provides the primary acceptance baseline.

Table 11: Symbolic Constants for System Tests

Constant	Value	Description and Requirements
FPS_MIN	72 FPS	Minimum average rendered frame rate in standalone mode (NFR-PERF-01).
T_FPS	120 s	Duration of each rendering performance test (NFR-PERF-01).
T_INPUT	100 ms	Maximum input-to-state-update time for qualifying events (NFR-PERF-02).
N_INPUT	100	Number of input events evaluated during response testing (NFR-PERF-02).
P_INPUT	95%	Minimum proportion of input events meeting the response-time threshold (NFR-PERF-02).
N_RECOVERY	20	Consecutive crash-and-respawn cycles per required obstacle-course environment (NFR-REL-01).
N_ENV	2	Minimum number of distinct virtual environments (FR-LVL-04).
N_LEVEL	3	Minimum number of selectable obstacle-course levels (FR-LVL-03).
N_CHECKPOINT	2	Minimum number of ordered checkpoints per obstacle course (FR-CHK-01).
S_BASE	1000	Initial score used in the performance calculation (FR-PER-08).
P_CRASH	100	Score penalty for each registered crash (FR-PER-08).
S_GOLD	800	Minimum score for a Gold rating (FR-PER-09).
S_SILVER	600	Minimum score for a Silver rating (FR-PER-09).
S_BRONZE	400	Minimum score for a Bronze rating (FR-PER-09).

The numerical score is defined in FR-PER-08 as

$$S = \max(0, S_{\text{BASE}} - \lfloor T \rfloor - P_{\text{CRASH}} C),$$

where T is the accumulated active flight time in seconds and C is the number of registered crashes. Ratings are assigned using the thresholds in Table 11, with scores below S_BRONZE classified as Needs Improvement.

Additional parameters from the Hazard Analysis and the planned validation studies are provided in Table 12.

Table 12: Additional Safety and Validation Parameters

Constant	Value	Description and Source
T_BREAK	30 min	Continuous-play duration before displaying a break reminder (Hazard Analysis SR21).
F_FLASH	3 per second	Maximum permitted number of flashes within any one-second period (Hazard Analysis SR22).
N_CRASH_SUG	3	Consecutive crashes within the same checkpoint section before suggesting free-flight practice (Hazard Analysis SR24).
N_BEGINNER	4	Target number of participants for the beginner interface study (Section 3.7).
N_PILOT	2	Target number of participants for the experienced-pilot realism study (Section 3.7).
B_TASK_PASS	3 of 4	Proposed minimum number of beginners completing each core task without assistance (Section 3.7).
B_RATING	4/5	Proposed minimum median beginner survey rating for each item (Section 3.7).
P_RATING	4/5	Proposed minimum rating from each experienced pilot for each realism item (Section 3.7).

The validation-study parameters represent proposed evaluation targets rather than acceptance requirements established by the SRS. These targets will be reviewed with the supervisor before participant testing.

The SRS also identifies proposed parameters for takeoff clearance, gravity verification, and performance-failure handling (PR-FLT-01, PR-FLT-02, and SR-04). Their numerical values remain subject to approval and will not be treated as finalized acceptance criteria until they are agreed upon and documented.

6.2 Usability Survey Questions

The usability evaluation aims to determine whether FPVR provides an intuitive, comfortable, and useful introductory drone training experience. The survey will assess the clarity of controller instructions, ease of navigation, perceived control responsiveness, effectiveness of training feedback, and overall

VR experience.

The survey supports the evaluation of NFR-USAB-01, NFR-SAFE-01, and NFR-SAFE-02, as well as related functional requirements concerning flight controls, training environments, crash recovery, and post-flight reviews. The results will contribute to the project's Usability Report, identified as a Custom Document in the Problem Statement.

6.2.1 Participants and Evaluation Procedure

Participants will be selected from the three intended user profiles identified in Section G.8 of the SRS: novice FPV pilots, casual gamers or technology enthusiasts, and experienced FPV pilots. The study will aim to include approximately 6–9 participants, with representation from each profile where feasible.

Each participant will receive a brief explanation of the simulator, its limitations, and the evaluation procedure. Participation will be voluntary, and participants may stop the session at any time if they experience discomfort. The headset will be used in a clear, stationary testing area, with a team member supervising the session.

Participants will be asked to complete the following activities:

1. Locate the control guide and identify the throttle, pitch, roll, and yaw controls.
2. Enter free-flight mode, practise basic drone movement, and return to the main menu.
3. Select an introductory obstacle course, navigate checkpoints, and interact with the crash-recovery options if a crash occurs.
4. Complete an obstacle course where feasible and review the recorded performance information, including completion time, crash count, score, and rating.

The team will observe whether participants complete tasks successfully, request assistance, encounter navigation difficulties, or experience discomfort.

These observations will supplement the survey responses.

6.2.2 Participant Background

Before beginning the evaluation, participants will answer the following questions to help interpret their feedback in relation to their experience.

Q1. How much previous experience do you have operating FPV drones?

Options: No experience, Beginner, Intermediate, Experienced.

Q2. How familiar are you with virtual reality headsets and VR applications?

Options: Not familiar, Slightly familiar, Moderately familiar, Very familiar, Extremely familiar.

Q3. Which controller configuration did you use during the evaluation?

Options: Meta Quest motion controllers, Xbox-compatible controller.

The execution mode (standalone VR or PCVR) will also be recorded by the evaluator.

6.2.3 Usability Evaluation Questions

Participants will rate the following statements on a five-point Likert scale:

1 = Strongly Disagree, 2 = Disagree, 3 = Neutral,
4 = Agree, 5 = Strongly Agree

Participants may select *Not Applicable* for features they did not experience.

Controller Interaction and Flight Experience

Q4. The instructions clearly explained how to use the throttle, pitch, roll, and yaw controls.

Q5. The drone responded to my controller inputs without noticeable delay.

Q6. The drone's movement was predictable and easy to understand while practising.

Q7. The first-person VR view made it easy to understand the drone's position and direction of movement.

Navigation and Training Experience

Q8. I could easily navigate the menus and select the training mode or level I wanted.

Q9. The obstacle-course objectives and checkpoint progression were easy to understand.

Q10. The difference between restarting a level and respawning at the last completed checkpoint was clear.

Q11. The free-flight mode provided a useful environment for becoming familiar with the controls before attempting obstacle courses.

Performance Feedback

Q12. The post-flight review clearly communicated my performance through completion time, crashes, score, and rating.

Q13. The performance feedback helped me identify areas where I could improve my next attempt.

VR Comfort and Safety

Q14. I was able to operate the simulator comfortably while remaining seated or standing in one location.

Q15. The pause and exit options were easy to find and use without removing the VR headset.

Q16. The simulator clearly communicated that success in virtual training does not guarantee the ability to safely fly a real drone.

Overall Experience

Q17. Overall, FPVR provided an understandable and useful environment for introductory drone flight practice.

6.2.4 Additional Feedback Questions

The following questions will collect qualitative feedback that may not be captured by the numerical ratings.

Q18. Did you experience any discomfort during the VR session, such as dizziness, nausea, eye strain, or disorientation? If so, please describe it. Participants may choose not to answer.

Q19. Which aspect of the simulator did you find most helpful for learning or practising drone controls?

Q20. What difficulties or confusing interactions did you encounter while using the simulator?

Q21. What changes would most improve your overall training experience?

Q22. For participants with previous FPV drone experience: How did the simulated drone's behaviour compare with your experience operating a physical drone? Were any movements or control responses noticeably unrealistic?

6.2.5 Evaluation and Analysis

The results of the survey will be analyzed using both quantitative and qualitative methods. For each Likert-scale question, the team will calculate the average rating and summarize the distribution of responses. Responses marked

Not Applicable will be excluded from the corresponding numerical calculations.

Results will be grouped by participants' prior FPV drone experience where the sample size permits. This will help determine whether beginners and experienced pilots encounter different usability problems or have different expectations regarding flight realism and training feedback.

The team will also record task completion, required assistance, observed navigation errors, and reported discomfort. Open-ended responses will be reviewed to identify recurring usability concerns and suggestions for improvement.

The findings will be documented in the Usability Report and used to inform changes to the controller guide, interface navigation, training environments, crash-recovery interactions, and performance feedback. Due to the limited number of participants, the results will be treated as formative usability evidence rather than definitive proof of improved real-world drone piloting ability.

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

- Claude Opus 5.5, Anthropic
- GPT-6 Astra, OpenAI

7.2 How and Where Used

1. **How were the AI tools used?**

   Claude Opus 5.5 and GPT-6 Astra were used as drafting, reviewing, and consistency-checking aids. GPT-6 Astra was used mainly to brainstorm section structure, propose candidate tests and survey questions, and rephrase technical writing so it followed the course template fields (control, initial state, input, expected output, derivation, and how the test will be performed). Claude Opus 5.5 was used mainly to turn the team's existing SRS, Hazard Analysis, Development Plan, and Problem Statement into draft paragraphs, to keep requirement identifiers and module names consistent across sections, and to apply the same test layout repeatedly. Both tools were also used to compare this plan against the template and to flag missing subsections, unmatched requirement identifiers, unused template comments, and wording that treated a planned activity as a completed result.

   The tools did not choose the product scope, the two custom documents, the testing priorities, or the out-of-scope items. Those decisions were made by the team. AI-assisted text was written to match them, and unused suggestions were discarded.

2. **Which parts of the document were affected by AI assistance?**

   - **Summary:** GPT-6 Astra produced a first-pass overview of FPVR as the software under test. Claude then rewrote it against the Problem Statement and SRS so the summary named the actual product functions (VR FPV training, free flight, obstacle courses, checkpoints, performance metrics, accounts, and leaderboards) rather than generic simulator features.

- **Objectives:** GPT-6 Astra suggested candidate VnV qualities (correctness, reliability, usability) and a list of possible exclusions. Claude drafted the in-scope objective as verifying FPVR's own implementation against the SRS, with Hazard Analysis used to rank mission-critical behaviour, and wrote the out-of-scope list (long-term training effectiveness, other VR headsets, and Unity or vendor headset software). Those exclusions were then checked against the Development Plan.
- **Custom Documents:** Claude drafted the descriptions of the Usability Report and the Design Thinking Report after the team confirmed those two custom documents from the Problem Statement. The draft states what each report will contain (task-based usability evaluation with surveys; interviews, empathy maps, ideation, and two prototype rounds) and why they belong in this VnV plan rather than as unrelated extras.
- **Relevant Documentation:** Claude drafted the explanation of how the SRS supplies goals, functional and non-functional requirements, and identifiers for traceability, and how the Hazard Analysis supplies failure modes and safety requirements that add invalid-input, tracking-loss, and unrealistic-physics tests. Claude also drafted the BibTeX entries that point at the team's SRS and Hazard Analysis PDFs; the URLs were supplied by the team.
- **Plan sections (team, SRS verification, design verification, VnV-plan verification, implementation verification, automated tools, and software validation):** Both tools were used to check that these subsections still follow the template (roles rather than a name list.
- **System tests:** GPT-6 Astra was used to brainstorm groupings of functional tests (flight, courses, menus, recovery, accounts) and non-functional tests (performance, usability, safety) from the SRS. Claude was used to keep each test in the template format and to attach requirement identifiers. Happy-path cases were taken from the SRS; failure cases (invalid controller input, lost tracking, crash recovery) were taken from the Hazard Analysis.
- **Traceability:** Claude was used to draft or check mappings from test identifiers to SRS requirement identifiers and from unit tests

to the nine logical modules in SRS section S.1, so that a requirement was not left with no planned verification.

- **Unit Test Description:** Claude drafted the introduction, the in-scope and out-of-scope module lists, the black-box / white-box / mocking strategy, and the named automated tests for Flight and Collision, Modes and Levels, Checkpoint Progress and Recovery, Timing and Evaluation, VR Interface and Input, Post-Flight Review, Accounts and Online Service, Results and Leaderboards, and Intelligent Feedback. GPT-6 Astra was used as a second pass to suggest extra edge cases (for example repeated contact producing one crash event, or no crash before takeoff).
- **Usability survey appendix:** GPT-6 Astra helped brainstorm Likert items and background questions. Claude drafted the participant procedure (approximately 6–9 people from the three SRS user profiles), the task script (control guide, free flight, course and crash recovery, post-flight review), the five-point rating items, qualitative follow-ups, and the analysis plan.
- **Wording and consistency review:** Both tools were used throughout to shorten sentences, to keep terminology aligned (requirement identifiers, module names, "system test" versus "unit test," stretch goals versus required tests), and to remove leftover template comments from sections the team had already filled.
- **This disclosure:** Claude Opus 5.5 drafted this appendix from the course template and from the team's record of which sections were AI-assisted. GPT-6 Astra was not used to invent a list of uses that did not occur.

AI-generated suggestions were used as drafts and review notes, not as final text. No figures or diagrams in this document were generated by AI. Citations were not invented by AI.

7.3 Verification of AI-Generated Content

1. **How the team checked AI-generated or AI-assisted material before including it in the document.**

Every AI-assisted change was reviewed by the author of that section before it was committed, and again through pull-request review by another team member before it was merged. Drafts were compared with their sources rather than accepted because they sounded complete: the summary and custom documents against the Problem Statement, objectives and tooling against the Development Plan, system and unit tests against SRS identifiers, and failure and safety tests against the Hazard Analysis. When a tool proposed a new feature, a numeric target, or a change of scope, the team confirmed the decision independently. Several suggestions were rejected for that reason, including treating Unity, OpenXR, or Meta system software as in-scope for FPVR unit tests, treating stretch-goal multiplayer or dedicated transmitters as required tests, and stating headset-wide compatibility testing that the objectives section explicitly excludes. Agreement between Claude and GPT-6 Astra was not treated as verification.

2. **How the team verified factual claims, technical assertions, requirements, calculations, and references.**

   - **Requirements and traceability:** Test cases that name a requirement (for example FR-FLT-03, FR-FLT-05, NFR-USAB-01) were checked against the identifiers defined in the SRS. Tests that name a hazard or safety requirement were checked against the Hazard Analysis. If a draft test cited an identifier that does not exist, or described behaviour that the SRS marks as a stretch goal or likely change, the test was rewritten or marked as applying only if that requirement is in scope.
   - **Technical assertions:** Claims about Unity Test Framework, NUnit, Unity Code Coverage, Edit Mode and Play Mode tests, and the Microsoft C# coding standard were checked against the Development Plan and the vendor documentation for those tools. They are recorded as the team's planned tools, not as evidence that the tests have already been executed.
   - **Calculations and expected outputs:** Expected results in unit tests (for example one crash event from repeated contact, or a decrease in vertical velocity under zero throttle) were derived from the corresponding SRS rules and checked by a team member.

Where the flight model is still under review, tests state the direction of change rather than a fabricated numeric trajectory. Score, rating, and timing oracles will use the formulae in the SRS; they are not taken from an AI-produced example.

- **Usability procedure:** Participant counts, user profiles, and task scripts were checked against Section G.8 of the SRS and the Custom Documents section of this plan so that the survey cannot silently invent a different study. Likert items were reviewed to ensure they measure the intended qualities (control clarity, navigation, comfort, feedback) rather than implementation details the participant cannot observe.
- **References:** AI tools were not used to invent citations. Bibliographic entries for the SRS and Hazard Analysis were written by the team and point at the project repository. The document was compiled with `pdflatex` after substantive edits, and the log was checked for undefined citations and missing labels.
- **Plans versus results:** AI-assisted wording was edited so that this document remains a plan. Coverage targets, frame-rate or latency numbers, and usability scores are treated as quantities to be measured later and reported in the VnV Report, not as verified outcomes of the present draft.

3. **Who is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document.**

   The team is responsible for the accuracy, completeness, consistency, and appropriateness of this Verification and Validation Plan, whether a passage was written by a team member or drafted with AI assistance. Claude Opus 5.5 and GPT-6 Astra can miss requirements, invent identifiers, or describe tests the team cannot run with the time and hardware available. Human review against the SRS, Hazard Analysis, Development Plan, and course template is therefore the check that the plan is feasible, internally consistent, and appropriate for FPVR.

8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.
4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Ibrahim Tariq

1. **What went well?** Using the SRS and Hazard Analysis gave us a clear starting point for choosing tests. Dividing responsibilities helped me focus on flight behaviour and performance while connecting my work to the broader plan.

2. **Pain points and resolutions.** The main challenge was making flight realism and VR comfort testable. I addressed this by separating measurable physics and performance checks from user feedback. Conflicting acceptance thresholds still need agreement before those tests can establish a pass.

3. **Knowledge and skills needed.** I need stronger skills with the Unity Profiler to identify frame-time bottlenecks and measure input responsiveness on our target hardware. I also need practice designing repeatable physics tests for gravity, controller inputs, and collisions, including choosing meaningful tolerances so I can distinguish implementation errors from limitations of the flight model.

4. **Learning approaches and choices.** I can improve my profiling skills by studying Unity's documentation or analysing captures from controlled flight scenes. I will focus on analysing captures so I can connect frame-time spikes to specific behaviour. For physics testing, I can follow Unity Test Framework tutorials or write small Play Mode tests with fixed inputs. I will prioritise writing tests, using tutorials as references, because this gives me practice isolating variables and reproducing failures.

Sankalpa Chhetri Dhakal

1. **What went well while writing this deliverable?**

   Having our finalized SRS and Hazard Analysis gave us a good foundation for developing the V&V Plan since we already had a clear understanding of the system's requirements and potential risks. I primarily worked on the symbolic parameters and usability survey questions, which helped me think about how we would evaluate FPVR beyond

just checking whether the software works. I also spent time reviewing my teammates' sections, which was useful for identifying missing details and making sure our verification strategies aligned with the requirements. Overall, dividing the document into sections made the workload more manageable.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**

   One of the main challenges was that different sections of the V&V Plan were closely connected, making it difficult to complete everything independently. For example, my symbolic parameters depended on the system tests written by another teammate, while my usability survey needed to align with the validation procedures described earlier in the document. We also identified inconsistencies between our SRS and Hazard Analysis, particularly regarding performance thresholds and which features were considered core requirements. Rather than ignoring these differences, we documented them for further review and worked on aligning our testing procedures with the finalized SRS.

3. **What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project?**

   Our team will need to strengthen several technical and testing skills to properly evaluate FPVR. Ibrahim will need to develop experience validating drone flight physics and measuring VR performance using Unity's profiling tools. John will focus on testing controller mappings, headset tracking, and compatibility between standalone and PCVR configurations. Vikram will need to become familiar with automated unit and integration testing using Unity Test Framework and NUnit, especially for checkpoint progression and crash recovery. Arjun will focus on verifying scoring accuracy, data persistence, and authentication security. I will need to improve my understanding of usability testing, survey design, and user feedback analysis to determine whether our simulator is intuitive and effective for beginners.

4. **For each knowledge area and skill identified above, what are at least two approaches to acquiring the knowledge or mastering**

the skill? Which approaches will each team member pursue, and why?

Each team member can develop these skills through studying technical documentation and tutorials or through practical experimentation with our project. Ibrahim will combine research on drone dynamics with controlled Unity simulations to compare expected and actual flight behaviour. John will use official XR documentation and build small hardware integration prototypes to identify compatibility issues early. Vikram will learn from NUnit tutorials and practise implementing automated tests for game-state functions, while Arjun will study secure data-handling practices and test scenarios involving failed saves and invalid authentication. I plan to review usability testing methods from HCI coursework and conduct structured sessions with potential users. We chose these approaches because combining theory with hands-on testing will help us develop the necessary skills while directly improving FPVR.

John Cho

1. **What went well while writing this deliverable?**

   Having a detailed Software Requirements Specification (SRS) made it much easier to write the system tests for this verification plan. The SRS provided specific requirements, expected behaviours, constraints, and acceptance criteria that could be converted into test objectives and observable pass or fail conditions. Instead of having to decide from scratch what the system tests should cover, I could trace each test back to a requirement and use the requirement's wording to determine the inputs, the expected result, and the evidence that should be collected. This was especially helpful for tests involving controller mappings, headset tracking, installation, and interruption recovery, because the SRS clarified what the user should experience without forcing the test to depend on a particular implementation. The detailed requirements also made it easier to identify missing cases and to check that the system tests covered normal operation as well as important edge cases. I likely would've had a much harder time writing functional tests without a detailed SRS otherwise.

2. **What pain points did you experience during this deliverable, and how did you resolve them?**

   A major pain point was that several requirements were similar or partially overlapping. As a result, the same behaviour could appear relevant to more than one system test, which made it difficult to determine where each requirement should be referenced and how to link all of the requirements without duplicating tests unnecessarily. It was also easy for a test written for one feature to implicitly cover another feature without making that relationship clear in the traceability table. I resolved this by first categorizing and grouping related requirements, such as input and controller behaviour, headset and tracking behaviour, installation and compatibility, and interruption recovery. I then wrote the system tests category by category, identifying the primary requirements for each test and recording any additional requirements that were also covered. This made the test boundaries clearer and produced more complete and consistent traceability.

3. **What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project?**

   The team will need knowledge of Unity and C# development, VR development on the Meta Quest 3S with OpenXR, drone flight dynamics, level and environment design, machine learning (optionally), state-machine design, testing, and generally good game design. These skills are needed to verify that the simulator behaves correctly, provides responsive and usable VR interactions, represents flight realistically, supports coherent training environments, preserves consistent game states, and reports meaningful performance results.

4. **For each knowledge area and skill identified above, what are at least two approaches to acquiring the knowledge or mastering the skill? Which approaches will each team member pursue, and why?**

   The team can acquire these skills through a combination of structured study and hands-on implementation. For Unity and C#, one approach is to use Unity Learn tutorials and the Unity manual, while a second approach is to build small prototypes before implementing full features.

For VR development, the team can study Meta, OpenXR, and XR Interaction Toolkit documentation and sample projects, or test early builds on the Quest 3S while measuring tracking, input, frame rate, and latency. For flight dynamics, the team can read about quadcopter dynamics and the reference drone, or compare the simulator with established FPV simulators. Level design can be learned by studying existing FPV courses and by conducting teammate playtests. Machine-learning skills can be developed through model documentation and small experiments on recorded flight data. State-machine testing can be developed by documenting transitions formally and writing unit tests for each transition. Scoring and flight-data skills can be developed by studying scoring systems in existing simulators and by calibrating an initial scoring formula with recorded team flights.

I will primarily pursue the VR documentation and hands-on Quest 3S testing approaches because my responsibilities include headset, controller, installation, and interruption-recovery testing. I will also use small hardware-integration prototypes to find compatibility and recovery issues early. The other members will combine the approaches that best match their responsibilities: Ibrahim will study flight dynamics and compare controlled simulations, Sankalpa will study and playtest course designs, Vikram will document and unit-test state transitions, and Arjun will study existing scoring systems before calibrating the scoring and data-recording implementation with real runs. Combining research with practical experiments will give the team both the background needed to design meaningful tests and direct evidence that the implementation satisfies the requirements.

Arjun Sasidaran

1. **What went well?** Splitting the plan into one GitHub issue per subsection at the start made it clear who owned each part and what each part had to contain. For the unit tests, I chose the nine modules early, based on the logical responsibilities in SRS S.1, so that the tests would not need to be rewritten once the MG and MIS are done. Comparing our draft against plans from previous capstone teams also helped me judge how much detail belongs in a plan and how much belongs in the

tests.

2. **Pain points and resolutions.** My first unit test drafts crossed module boundaries. For example, one Respawn test checked checkpoint progress, timers, and crash count at the same time, which made it an integration test. I fixed this by making each test check one module and replacing the others with fakes, and by moving the timer and crash checks into the Timing and Evaluation tests. A similar problem appeared when reviewing the system tests: several tests cited requirements that their expected output never showed. I split them so that every test's output demonstrates each requirement it names, and kept functional and nonfunctional requirements in separate tests. Finally, with several people editing the same LaTeX file, merge conflicts were common. Keeping each branch to one section and checking it against the open pull requests before pushing reduced this.

3. **Knowledge and skills needed.** I need to learn security and privacy testing for the account and online result service. This includes confirming that credentials are only sent over encrypted connections and never stored in plaintext, that one player cannot read or submit another player's data, and that FPVR only sends and keeps the data it needs. These checks are new to me, and they are the evidence for the security tests in Section 4.2.

4. **Learning approaches and choices.** One approach is to study the authentication, session, and access-control sections of the OWASP Web Security Testing Guide and the documentation of the online service we choose. Another is hands-on practice: capturing my own test traffic with a proxy such as mitmproxy, inspecting local storage, and writing automated access-control tests against a test account. I will focus on the hands-on approach and use OWASP as a checklist. The online provider is still undecided, so practising against a test service lets me build the checks now and reuse them once the provider is chosen.

Vikram Chandar

1. **What went well?** Writing the General Information section first, including the summary, objectives, custom documents, and relevant doc-

umentation, gave the team a shared description of what FPVR is and what this plan covers. Later sections could refer back to it instead of restating the scope. Because the SRS and Hazard Analysis were already complete, I could base each part directly on them.

2. **Pain points and resolutions.** While reviewing the functional system tests, I found tests that cited requirements they did not actually exercise. For example, the free-flight test claimed the free-flight crash recovery requirement without ever crashing the drone, and one menu test claimed several level requirements just by loading a course. I raised these in the pull request review so the tests could be split into more specific ones. It was also difficult to keep the General Information section short while still stating clearly what is out of scope, so I kept it to the points the rest of the plan depends on.

3. **Knowledge and skills needed.** I need to learn state-transition testing for the game state: the attempt states, checkpoint progress, crashes, Respawn, Restart Level, and the timers. Most of the rules I am responsible for depend on the order of events, such as a crash before the first checkpoint, so testing single values is not enough. I also need to learn how to write these tests in the Unity Test Framework so they run automatically.

4. **Learning approaches and choices.** One approach is to study state-transition testing as a technique, for example through the ISTQB Foundation syllabus or the coverage criteria in Ammann and Offutt's *Introduction to Software Testing*. Another is to model the attempt states as a diagram and write one Edit Mode test for each transition, including invalid ones. I will start by building the state model and then write the tests from it, because the model shows which transitions are missing tests and gives the team something to review before the code is written.